

RISC-V AXI UART SoC

RTL Design, Verification, Synthesis & ASIC Physical-Design Exploration

PROJECT REPORT — WORKING DRAFT

Student Name: CHANDANA SRI SUNKARA

Department: Electronics and Communication Engineering

Institution: RGUKT, SRIKAKULAM

Program: B.Tech

Year: 4th year

Academic Year: 2026-2027

This draft is intended to be edited with final student/institution details, project dates, references, screenshots and acknowledgements.

Document Scope and Accuracy

This report records the work genuinely performed in the project. The open-source PicoRV32 processor core is treated as pre-existing IP integrated into the SoC; the project does not claim implementation of the PicoRV32 CPU itself.

The project work covered RTL integration, AXI/RAM/UART modules, testbenches, RISC-V firmware compilation and memory initialization, Verilator simulation, GTKWave waveform inspection, Yosys synthesis, synthesized schematic analysis, and an OpenLane/SKY130A physical-design run.

The OpenLane run completed synthesis, timing analysis, floorplanning, I/O placement, power planning, placement and clock-tree synthesis. Routing stopped because of high congestion. A successful final routed GDS is therefore not claimed.

Contents

1. Abstract and Objectives
2. System Overview and Architecture
3. Theoretical Background
4. RTL Implementation
5. Firmware and Memory Initialization
6. Verification and Waveform Analysis
7. Synthesis and Schematic Analysis
8. OpenLane Physical-Design Flow
9. Results, Limitations and Future Work
10. Attribution and References.

1. Abstract and Objectives

This project presents a small RISC-V based System-on-Chip (SoC) implemented in Verilog RTL. The SoC integrates the open-source PicoRV32 processor core with an AXI-based interconnect, on-chip RAM and a memory-mapped UART. The main demonstration is a firmware-driven UART transmission: software writes 0x41 to the UART transmit register at address 0x10000000, causing the CPU to generate an AXI write that is routed to the UART.

The design was verified first at module level and then at complete-SoC level. Verilator was used for simulation, GTKWave for waveform inspection, and Yosys for generic RTL synthesis. The design was subsequently explored using OpenLane/OpenROAD with the SKY130A PDK. The physical-design run progressed through clock-tree synthesis but encountered routing congestion.

Objectives

- Integrate an open-source RISC-V core into a small SoC.
- Understand AXI-Lite channels, handshakes and address decoding.
- Connect RAM and UART as memory-mapped targets.
- Verify individual modules and the complete CPU-to-UART path.
- Compile and load a RISC-V firmware program into RAM.
- Perform synthesis, schematic inspection and an ASIC physical-design exploration.

2. System Overview and Architecture

The project is best understood as a set of cooperating blocks rather than as one large Verilog file. The following diagram is the main architecture figure for the report.

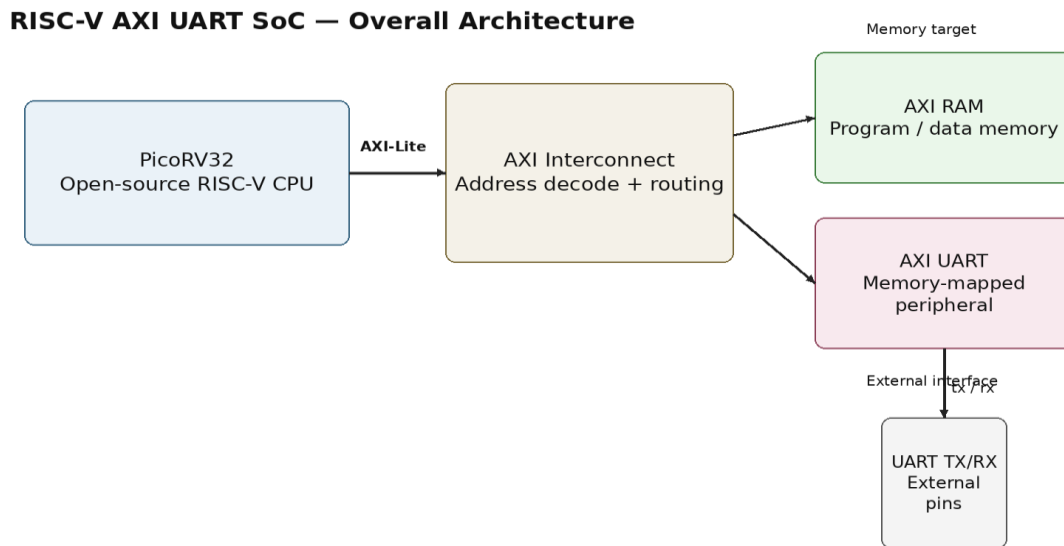


Figure 1. Project-specific overall SoC architecture.

The CPU is internal to the SoC and communicates through the AXI interconnect. RAM is used for the program/data image. The AXI UART provides the register interface to the UART transmitter and receiver. The UART pins are the only functional peripheral signals exposed at the top level.

2.1 Module Hierarchy

Module	Purpose
soc_top.v	Top-level integration and internal AXI wiring.
picoRV32a.v	Open-source PicoRV32 RISC-V core used as integrated IP.
axi_interconnect.v	Address decode, routing and response selection.
axi_ram.v	AXI-connected RAM and firmware initialization.
axi_uart.v	AXI register interface around UART TX/RX.
uart_tx.v	Serial transmitter.
uart_rx.v	Serial receiver.

The screenshot shows the Visual Studio Code interface with a project named 'riscv_axi_soc'. The Explorer on the left displays the file hierarchy: 'riscv_axi_soc' (containing 'firmware', 'ip' with 'picoRV32a.v', 'obj_dir', 'openlane' with 'soc_top', 'src' with 'config.json', 'rtl' with 'axi_interconnect.v', 'axi_ram.v', 'axi_uart.v', 'ram.v', and 'soc_top.v' (selected)), 'uart_rx.v', 'uart_tx.v', 'sim', 'sw', 'synthesis', 'tb', 'axi_interconnect.vcd', 'axi_ram.vcd', 'axi_uart.vcd', 'ram.vcd', 'Outline', and 'Timeline'. The main editor shows the 'soc_top.v' file with the following code:

```
1 timescale 1ns/1ps
2
3 module soc_top #(
4     parameter CLK_FREQ = 10_000_000,
5     parameter BAUD_RATE = 1_000_000
6 )(
7     input wire clk,
8     input wire resetn,
9
10    // External UART pins
11    input wire uart_rx,
12    output wire uart_tx,
13    output wire uart_tx_busy
14 );
15
16 // PicoRV32 AXI-lite master signals
17 wire cpu_avvalid;
18 wire cpu_awready;
19 wire [31:0] cpu_awaddr;
20
21 wire cpu_vvalid;
22 wire cpu_wready;
23 wire [31:0] cpu_wdata;
24 wire [3:0] cpu_wstrb;
25
26 wire cpu_bvalid;
27 wire cpu_bready;
28 wire [1:0] cpu_bresp;
29
30 wire cpu_arvalid;
```

The right sidebar shows a 'Chat' panel with a 'Build with Agent' section, indicating AI responses may be inaccurate and suggesting to generate agent instructions to onboard AI onto the codebase. The bottom status bar shows 'Ln 1, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', 'Plain Text', and the system clock '14:26 26-08-2026'.

Figure 2. Project RTL hierarchy and top-level [soc_top] implementation in VS Code.

2.2 PicoRV32 Integration

Only the processor behavior relevant to this SoC is required for the project explanation. The CPU contains the program counter, register file and execution logic, while its memory-facing interface is adapted for the AXI-

connected SoC.

PicoRV32 Integration Architecture

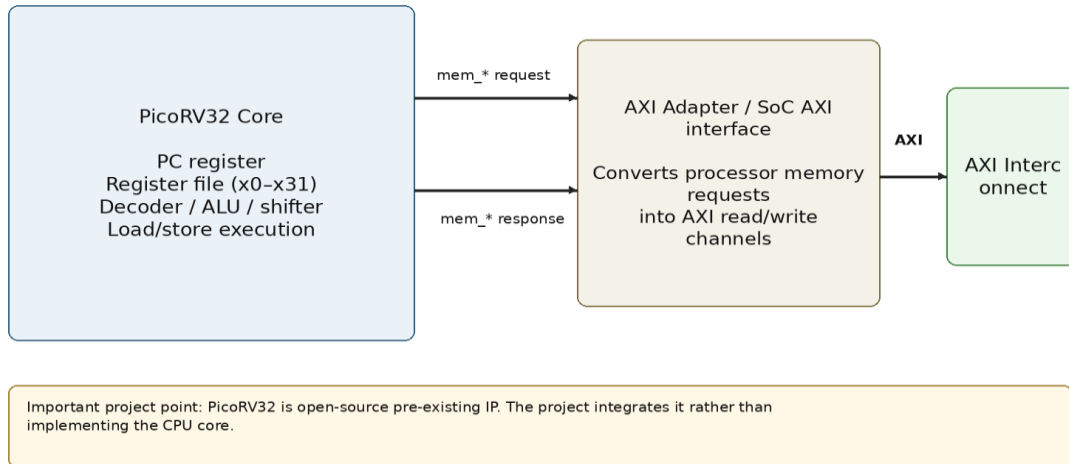


Figure 3. Project-specific view of PicoRV32 integration with the SoC.

Important attribution: PicoRV32 is open-source pre-existing IP. The project contribution is its integration with the AXI interconnect, RAM and UART subsystem, along with verification and implementation.

2.3 AXI-to-Peripherals Architecture

The interconnect provides the routing boundary between the processor and the two main targets. The high-level decision is simple: an address in the RAM region selects RAM, while an address beginning with hexadecimal 1 in the upper nibble selects UART.

```
assign addr_is_ram = (m_awaddr[31:28] == 4'h0);  
assign addr_is_uart = (m_awaddr[31:28] == 4'h1);
```

For example, 0x10000000 has `m_awaddr[31:28] = 4'h1`, so it is routed to the UART side.

Memory Map and Memory-Mapped I/O

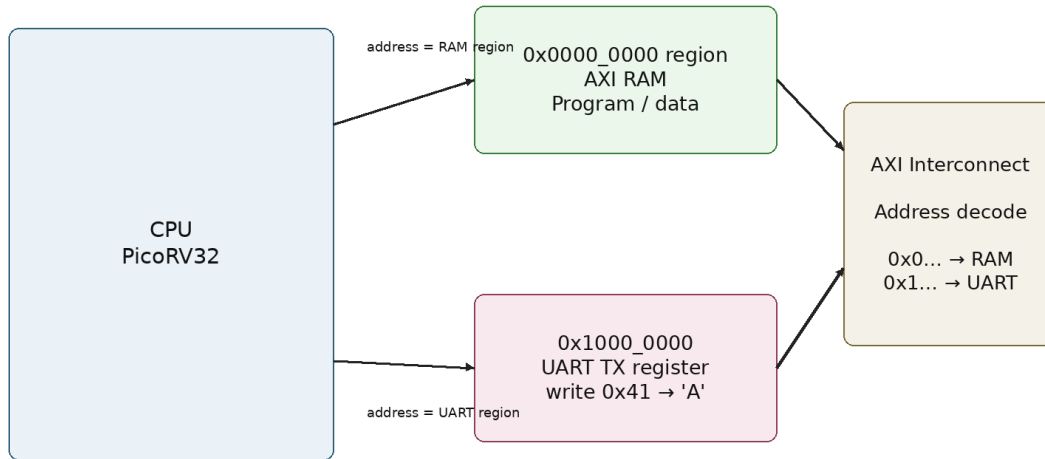


Figure 4. Memory map and memory-mapped I/O used in the project.

3. Theoretical Background

3.1 RISC-V and PicoRV32

RISC-V is an open instruction-set architecture (ISA). An ISA specifies the programmer-visible instruction set, registers and architectural behavior of a processor. PicoRV32 is a compact open-source RISC-V processor core. In this project, it is integrated as pre-existing IP.

- Program counter: selects the next instruction address.
- Register file: holds architectural general-purpose registers.
- Decoder and execution logic: interprets and performs instructions.
- Memory interface: issues instruction/data accesses used by the SoC integration.

The important system-level relationship is: a software store instruction can become a memory write request, which is then presented on the CPU's AXI-facing interface.

3.2 AXI / AXI-Lite

AXI is an AMBA on-chip interconnect protocol. AXI-Lite provides the essential channel structure without the burst-oriented complexity of full AXI4 and is well suited to memory-mapped register and control accesses.

AXI-Lite Channel Architecture

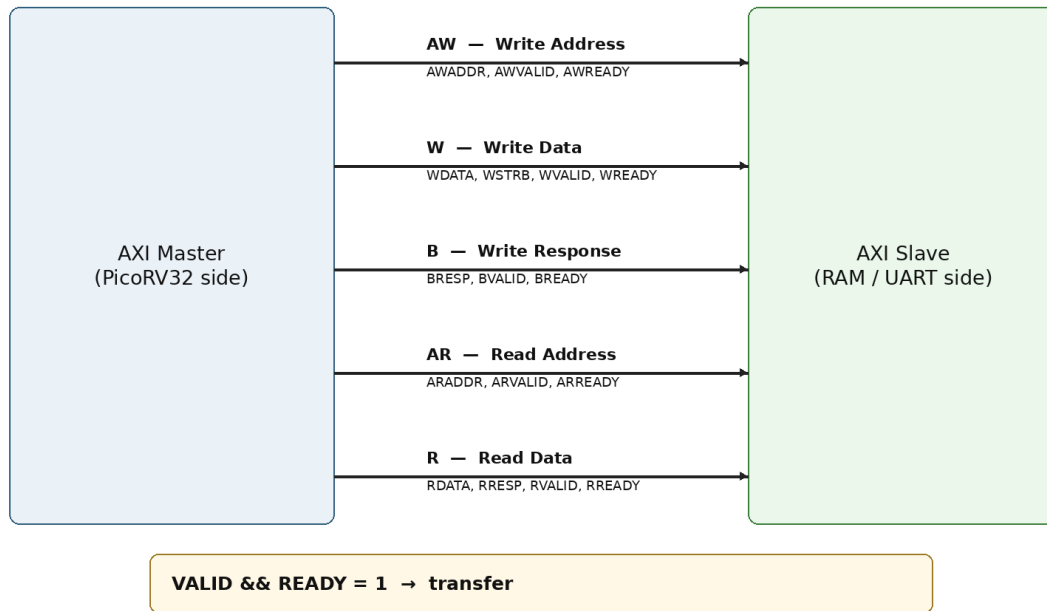


Figure 5. AXI-Lite five-channel architecture and VALID/READY handshake.

Channel	Purpose	Signals used in this project
AW	Write address	AWADDR, AWVALID, AWREADY
W	Write data	WDATA,WSTRB, WVALID, WREADY
B	Write response	BRESP, BVALID, BREADY
AR	Read address	ARADDR, ARVALID, ARREADY
R	Read response/data	RDATA, RRESP, RVALID, RREADY

For a transfer to occur on any AXI channel, the source must assert VALID and the destination must assert READY in the same clock cycle. The channels are logically independent, so address and data are carried separately.

```

Write acceptance:
AWVALID && AWREADY -> address transfer
WVALID && WREADY -> data transfer
BVALID && BREADY -> response transfer

```

3.3 UART

UART is an asynchronous serial interface. The endpoints agree on a baud rate and frame format instead of sharing a clock line. A common frame format is 8-N-1: one start bit, eight data bits, no parity and one stop bit.

In the project, the AXI UART receives register accesses and connects them to `uart_tx` and `uart_rx`. The transmitter serializes the byte and exposes it on the external tx pin; the receiver samples the external rx pin and reconstructs the received byte.

AXI UART Internal Architecture

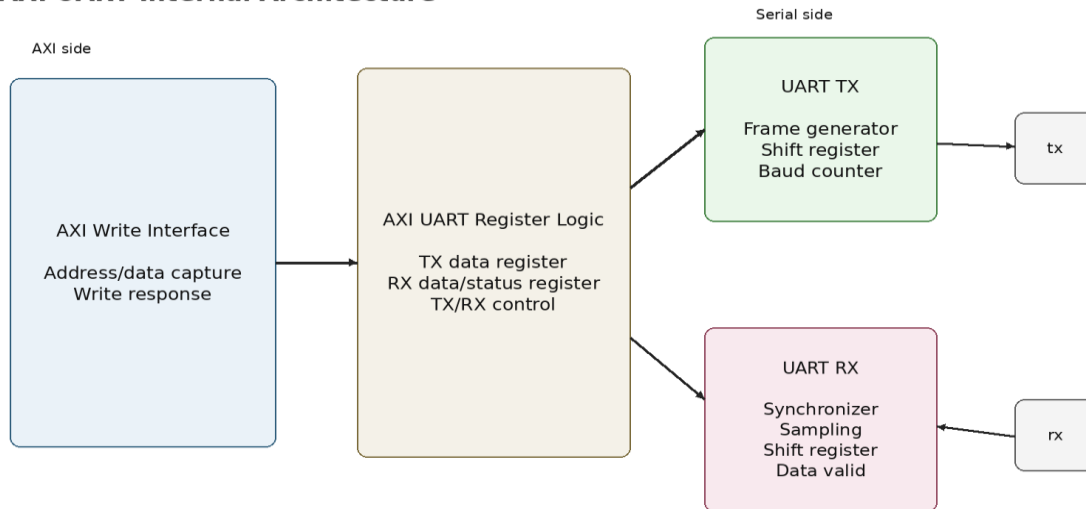


Figure 6. AXI UART internal architecture.

3.4 UART Frame and LSB-First Data

Idle	Start	D0	D1	D2	D3	D4	D5	D6	D7	Stop	Idle
1	0	LSB -----> MSB							1		

For the project test value 0x41, the data bits are sent LSB first. The UART transmit waveform should therefore be interpreted as a serial representation of the byte rather than as the parallel hexadecimal value 0x41.

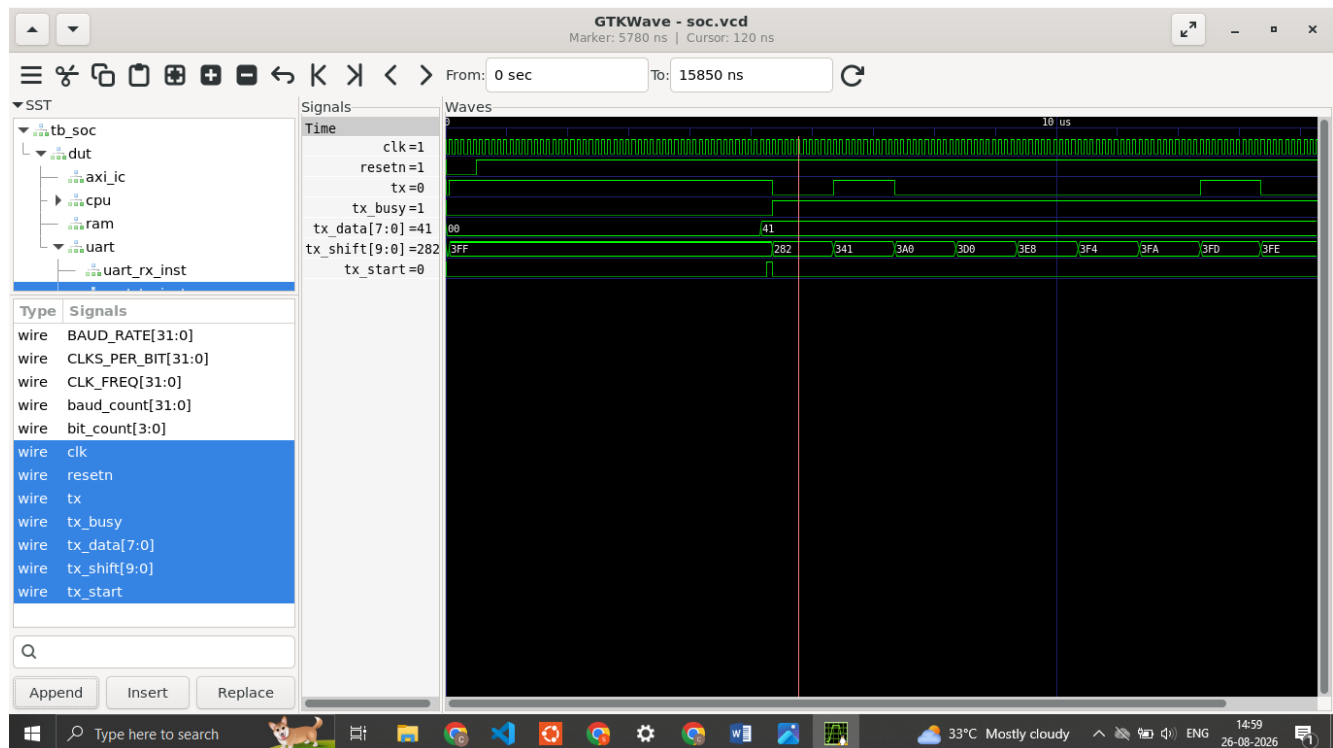


Figure 7. GTKWave showing UART transmission of 0x41, including tx_start, tx_busy, serialized TX data and UART internal shift register.

3.5 Memory and Memory-Mapped I/O

Memory-mapped I/O assigns addresses to peripheral registers. The CPU accesses the peripheral using ordinary load/store instructions; the interconnect uses the address to select the destination.

Address	Target / Use
0x00000000 region	SoC RAM / program memory region.
0x10000000	UART transmit register.
0x10000004	UART RX data register, used in the earlier AXI UART tests.
0x10000008	UART status register, used in the earlier AXI UART tests.

This address map links the firmware, the AXI interconnect and the UART registers into one coherent software-visible memory space.

3.6 AXI Interconnect: Address Decode and Response MUXing

At RTL, address decoding is compact. After synthesis, the same behavior expands into MUX, AND, OR, NOT and buffer cells. The synthesized interconnect schematic is therefore a lower-level implementation of the simple routing rules described above.

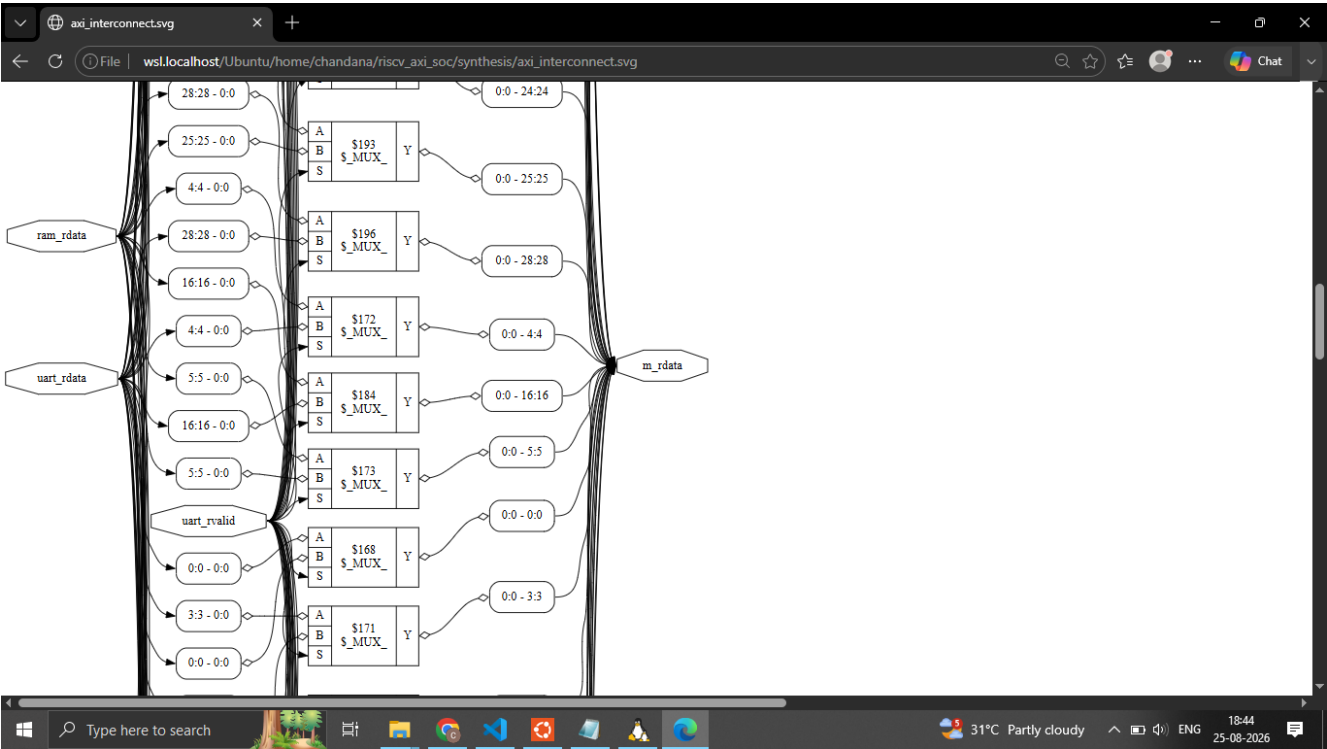


Figure 8. Synthesized AXI Address Decode and Response Multiplexing Logic.

4. RTL Implementation

4.1 soc_top

```
module soc_top #(
    parameter CLK_FREQ = 10_000_000,
    parameter BAUD_RATE = 1_000_000
) (...);
```

The top-level parameters establish the clock and UART configuration used in simulation. With a 10 MHz clock, the clock period is 100 ns. With a 1 MHz UART baud rate, one UART bit takes approximately 1 us.

4.2 AXI Interconnect

The interconnect copies the master address/data toward both targets but gates valid/ready signals using decoded address conditions. Responses are then selected from the active target.

```
assign ram_awvalid = m_awvalid && addr_is_ram;
assign uart_awvalid = m_awvalid && addr_is_uart;

assign m_awready =
    (addr_is_ram && ram_awready) ||
    (addr_is_uart && uart_awready);
```

The same principle is applied to write data and read paths.

4.3 AXI RAM

The RAM implements an AXI slave interface around an internal memory array. The project uses a word-oriented initialization file generated from the RISC-V firmware.

```
$readmemh("sw/uart_test_words.hex", mem);
```

This statement explains why the OpenLane environment also needed access to the firmware HEX file during synthesis: the RAM source contains a memory-initialization reference.

4.4 AXI UART

The AXI UART module captures writes to the transmit register, provides readback/status behavior and instantiates the UART TX/RX engines. The external top-level pins remain uart_tx and uart_rx.

4.5 UART TX and RX

The transmitter uses a state-machine-style sequence such as idle, start, data and stop timing. The receiver waits for the start edge, verifies the start-bit center, samples data bits, then checks the stop condition. The exact RTL implementation is contained in uart_tx.v and uart_rx.v.

5. Firmware and Memory Initialization

The firmware test demonstrates software-visible peripheral access. The key C operation is:

```
#define UART_TX 0x10000000

*(volatile unsigned int *)UART_TX = 0x41;
```

This is a memory-mapped store. The compiler turns it into RISC-V machine instructions. The linked ELF was disassembled to confirm the resulting instruction sequence, then converted into raw bytes and finally into a 32-bit word-oriented HEX file for RAM initialization.

Firmware-to-Hardware Flow

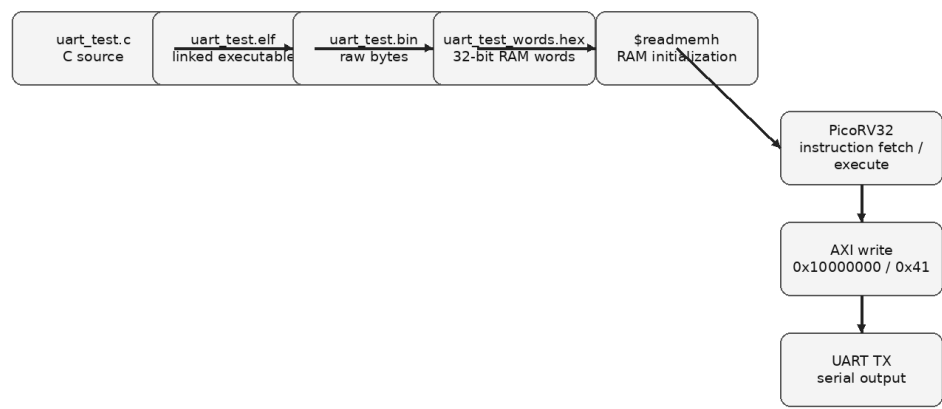


Figure 10. Firmware-to-hardware flow used in the project.

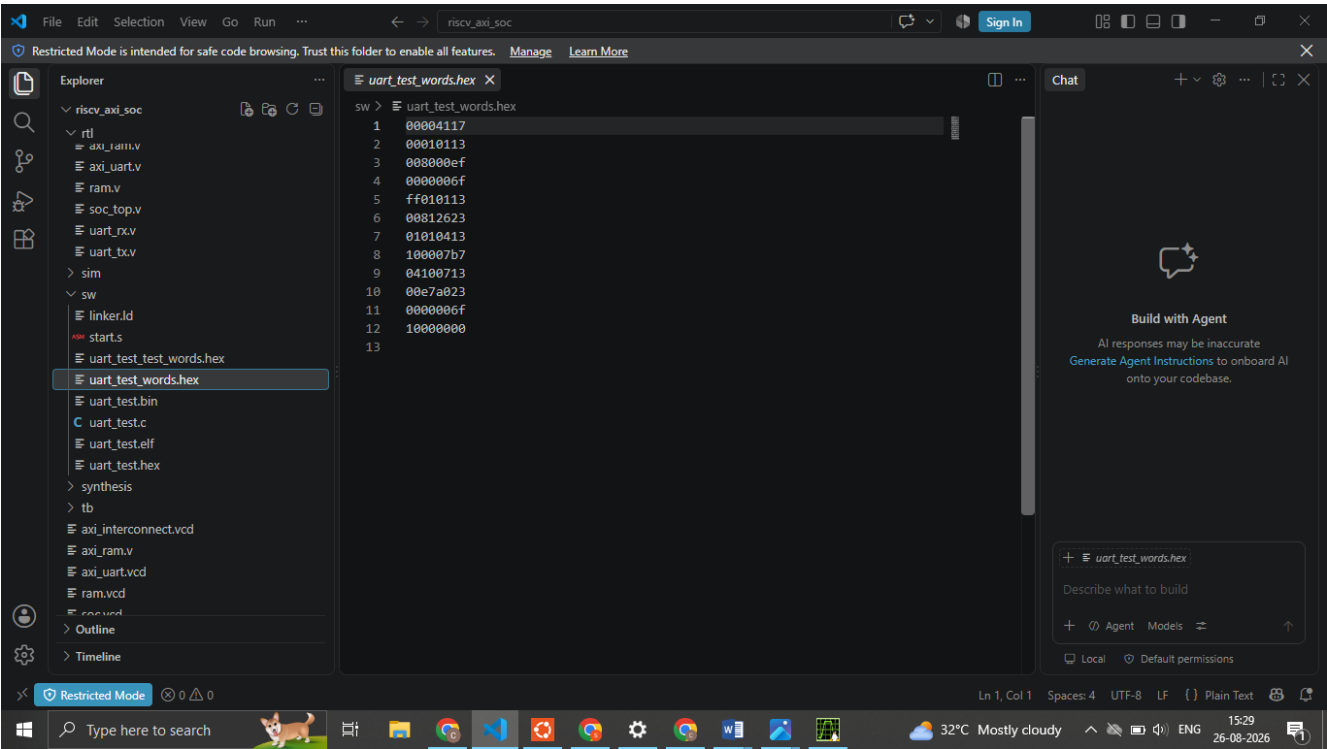


Figure 11. RISC-V firmware represented as 32-bit hexadecimal instruction words for RAM initialization.

6. Verification and Waveform Analysis

6.1 Testbench Hierarchy

Testbench	Primary purpose
tb_uart_tx.v	Verify UART TX behavior.
tb_uart_rx.v	Verify UART RX behavior.
tb_axi_uart.v	Verify AXI UART interface.
tb_axi_ram.v	Verify AXI RAM read/write behavior.
tb_axi.v	Verify AXI interconnect and RAM/UART routing.
tb_soc.v	Verify the integrated CPU→AXI→UART system.

6.2 Complete SoC Simulation Result

DUT: AW HANDSHAKE addr=10000000
DUT: W HANDSHAKE data=00000041
UART TX STARTED.
CPU successfully reached UART TX register.
UART TX COMPLETED.

RISC-V AXI UART TEST PASSED.

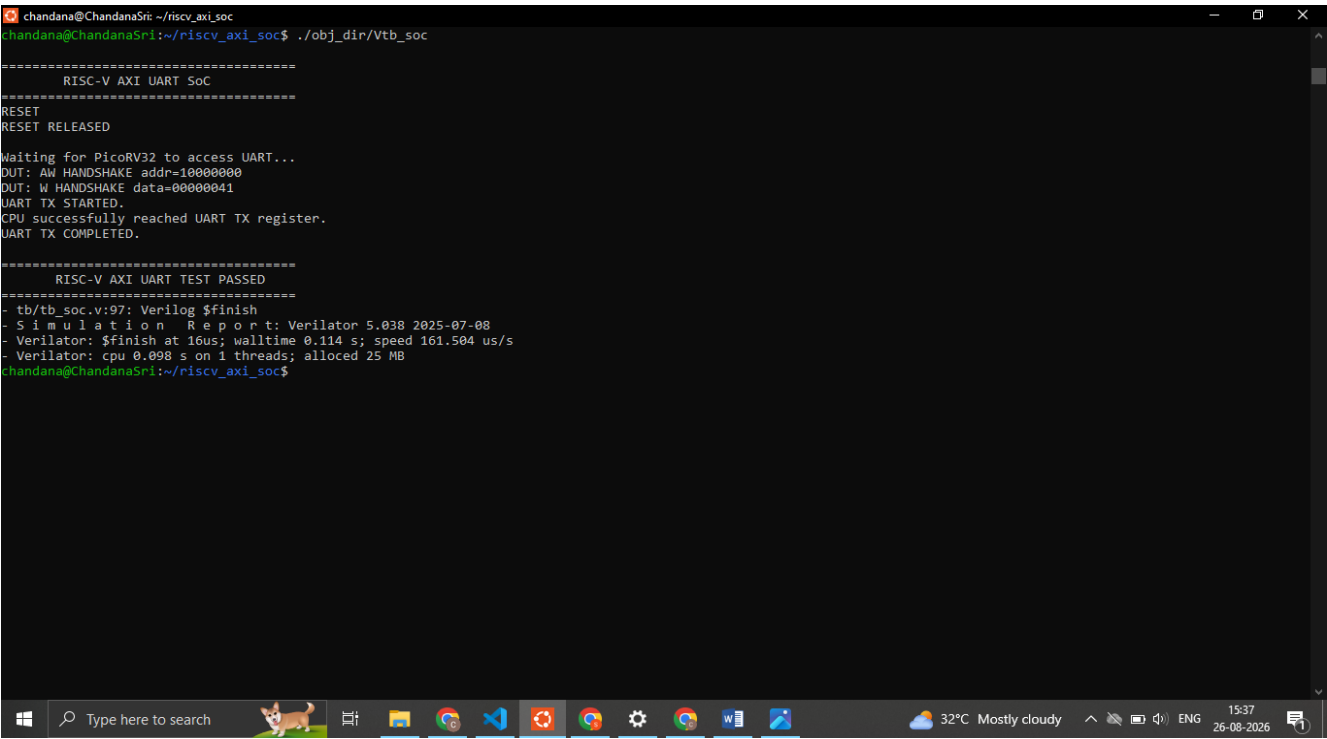


Figure 12. Complete RISC-V AXI-UART SoC Simulation Result

6.3 Waveform Evidence

The main waveform evidence should tell one continuous story rather than showing many unrelated signals.

- CPU AXI address and data handshakes.
- Interconnect master-side and UART-side valid signals.
- UART tx_busy transition.

- UART tx serial waveform.

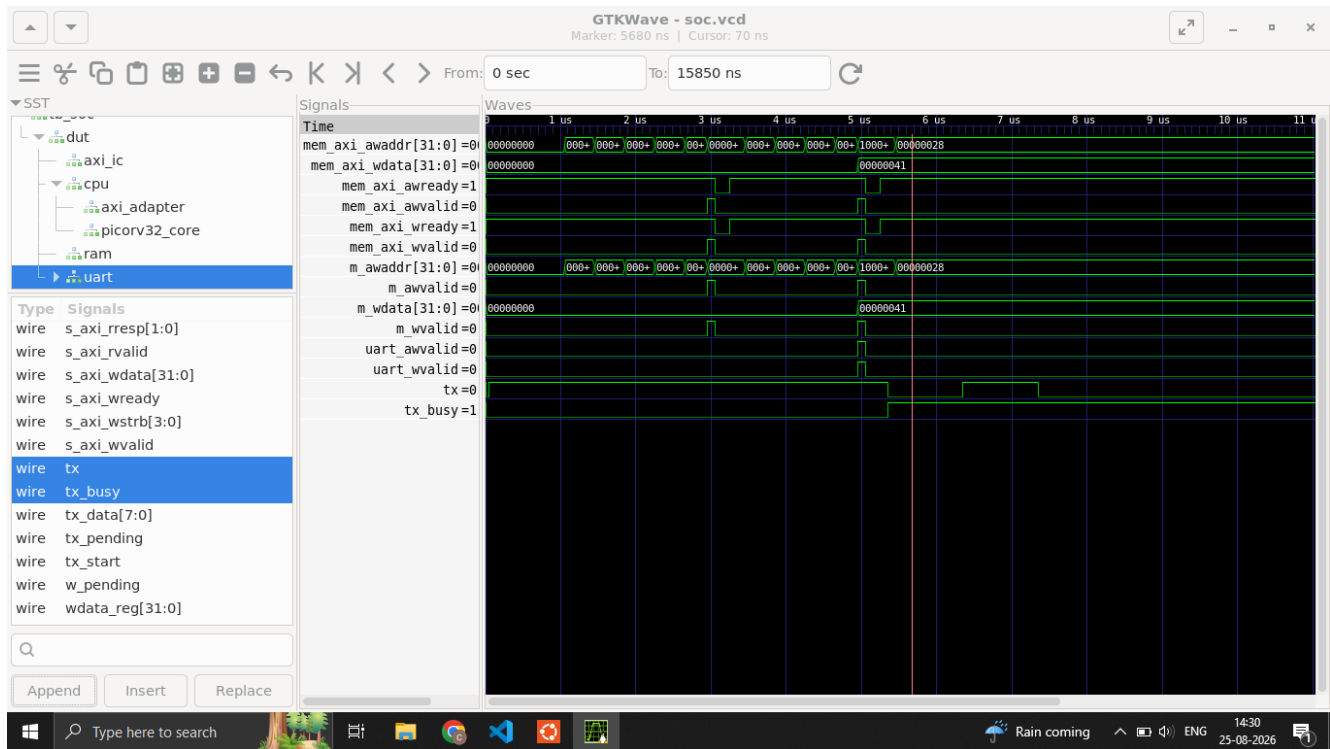


Figure 13. GTKWave Simulation Waveform of AXI and UART Transactions.

7. Synthesis and Schematic Analysis

7.1 Yosys Synthesis

The RTL was synthesized with Yosys. The generic synthesis flow produced a synthesized Verilog netlist and statistics. The complete design hierarchy was approximately 81,842 generic cells.

Module / hierarchy	Cells observed
Complete soc_top hierarchy	81,842
axi_ram	70,933
axi_uart	134
uart_rx	550
uart_tx	258
axi_interconnect	73

The AXI RAM dominates because the current memory implementation is represented as large amounts of standard-cell sequential and multiplexing logic in generic synthesis. This becomes important later in the physical-design discussion.

```

chandana@ChandanaSir: ~/riscv_axi_soc

=== axi_interconnect ===

Number of wires:      73
Number of wire bits:  464
Number of public wires: 57
Number of public wire bits: 444
Number of memories:   0
Number of memory bits: 0
Number of processes:  0
Number of cells:      73
$ _AND_                16
$ _MUX_                36
$ _NOT_                6
$ _OR_                 15

=== axi_ram ===

Number of wires:      7331
Number of wire bits:  71602
Number of public wires: 1048
Number of public wire bits: 32986
Number of memories:   0
Number of memory bits: 0
Number of processes:  0
Number of cells:      70933
$ _AND_                5225
$ _DFFE_PP_            32768
$ _MUX_                32840
$ _NOT_                15
$ _OR_                 3
$ _SDFFE_PN0P_         78
$ _SDFFE_PP0P_         4

=== picorv32_axi_adapter ===

Number of wires:      50
Number of wire bits:  278
Number of public wires: 30
Number of public wire bits: 257
Number of memories:   0
Number of memory bits: 0
Number of processes:  0
Number of cells:      31

chandana@ChandanaSir: ~/riscv_axi_soc

Number of cells:      31
$ _AND_                11
$ _DFFE_PP_            1
$ _MUX_                2
$ _NOT_                6
$ _OR_                 8
$ _SDFFE_PP0P_         2
$ _SDFFE_PP0P_         1

=== soc_top ===

Number of wires:      56
Number of wire bits:  443
Number of public wires: 56
Number of public wire bits: 443
Number of memories:   0
Number of memory bits: 0
Number of processes:  0
Number of cells:      4
$paramod$e42465b9e259ab0b6bc45fe1c5feb129e75f41e8\axi_uart 1
$paramod$picorv32_axi\PROGADDR_RESET=32'00000000000000000000000000000000 1
axi_interconnect      1
axi_ram               1

=== design hierarchy ===

soc_top 1
$paramod$e42465b9e259ab0b6bc45fe1c5feb129e75f41e8\axi_uart 1
$paramod$e42465b9e259ab0b6bc45fe1c5feb129e75f41e8\uart_rx 1
$paramod$e42465b9e259ab0b6bc45fe1c5feb129e75f41e8\uart_tx 1
$paramod$picorv32_axi\PROGADDR_RESET=32'00000000000000000000000000000000 1
$paramod$6ce030db3796b28030d2721a573c1331c27d5357\picorv32 1
picorv32_axi_adapter 1
axi_interconnect      1
axi_ram               1

Number of wires:      9910
Number of wire bits:  86955
Number of public wires: 1483
Number of public wire bits: 37495
Number of memories:   0
Number of memory bits: 0
Number of processes:  0
Number of cells:      81842

```

Figure 14 a,b. Yosys synthesis results showing module-level cell statistics and the overall synthesized SoC hierarchy.

7.2 Synthesized Schematic

The synthesized AXI interconnect shows how the RTL address-decoding and routing logic is implemented using logic gates and multiplexers. The main functions visible in the schematic are **address decoding**, **RAM/UART target selection**, **response multiplexing**, and **signal pass-through paths**.

The synthesized SoC schematic shows the major integrated blocks and their synthesized interconnections, including the **PicoRV32 CPU**, **AXI interconnect**, **AXI RAM** and **AXI UART**.

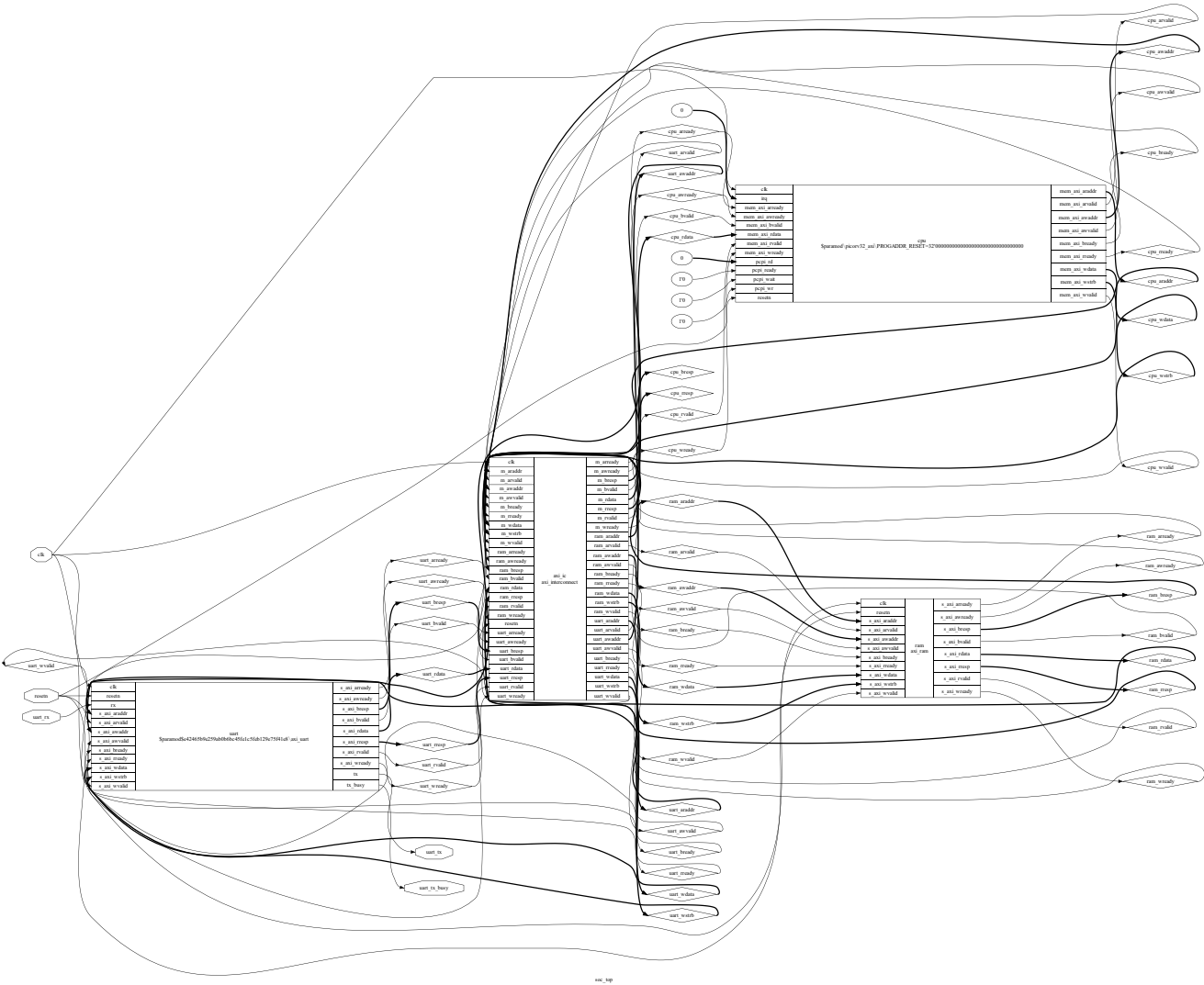


Figure 16. Synthesized Top-Level RISC-V AXI SoC Schematic.

8. ASIC Physical-Design Flow with OpenLane/SKY130

8.1 Why OpenLane Is Used

OpenLane is not another functional protocol used by the SoC. It is the implementation flow used after the RTL is functionally verified. Its job is to take the digital design toward physical layout using a PDK and standard-cell libraries.

OpenLane Physical-Design Run Stages — Project Result

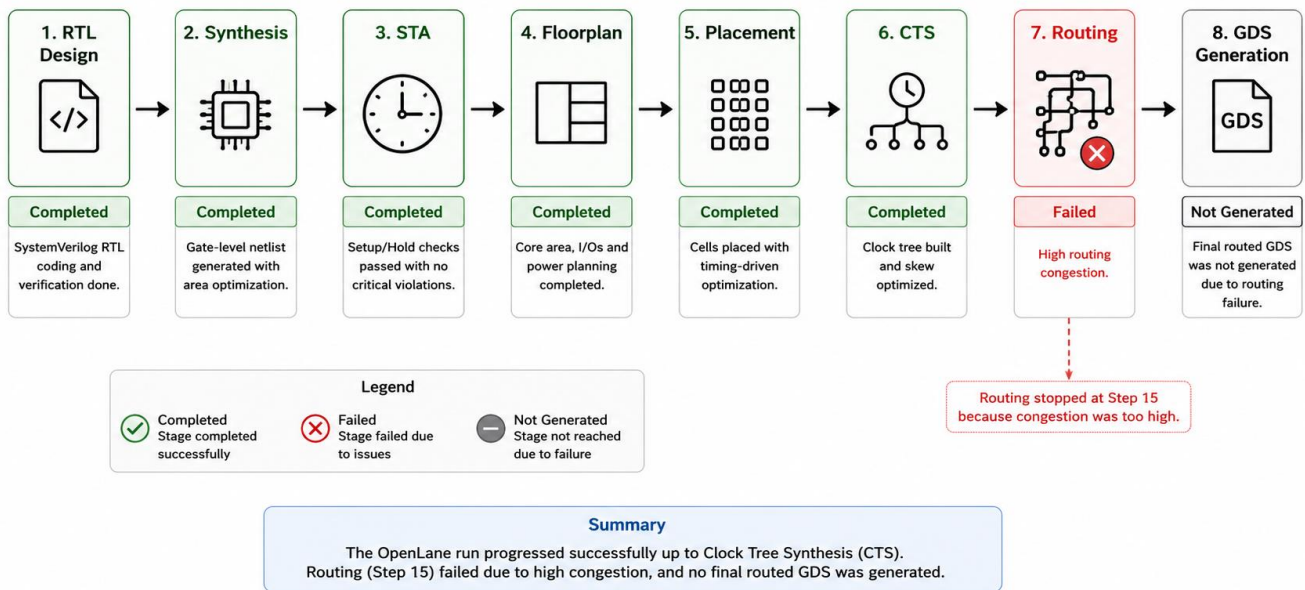


Figure 17. OpenLane flow stages reached by the project.

8.2 Where the Project Uses Physical-Design Concepts

Physical-design concept	Where it appeared in the project
Technology library	OpenLane selected SKY130A and the sky130_fd_sc_hd standard-cell library.
Synthesis	OpenLane synthesized the same RTL into a technology-aware implementation flow.
Static timing analysis	OpenLane ran STA after synthesis, placement and CTS.
Floorplanning	OpenLane generated a floorplan; the selected run reported width 2351.52 and height 2350.08.
I/O placement	Top-level pins were placed.
Tap/decap cells	Inserted as part of the floorplan stage.
PDN	Power planning and power distribution network generation were executed.
Placement	Global and detailed placement completed.
CTS	Clock Tree Synthesis completed.
Routing	OpenROAD attempted routing but stopped because congestion was too high.

8.3 The OpenLane Run in This Project

The actual run used OpenLane v1.0.2 inside a Docker container. The PDK root was /home/chandana/.ciel and the selected process was sky130A.

The flow succeeded through Step 14 (placement resizer/CTS-related processing). At Step 15, routing reported high congestion. The run therefore demonstrates the RTL-to-physical-design path up to CTS, but not a completed routed GDS.

```
Select chandana@ChandanaSir ~
[INFO]: PDK Root: /home/chandana/.ciel
[INFO]: Process Design Kit: sky130A
[INFO]: Standard Cell Library: sky130_fd_sc_hd
[INFO]: Optimization Standard Cell Library: sky130_fd_sc_hd
[INFO]: Run Directory: /openlane/designs/soc_top/runs/RUN_2026.08.26_05.09.32
[INFO]: Saving runtime environment...
[INFO]: Preparing LEF files for the nom corner...
[INFO]: Preparing LEF files for the min corner...
[INFO]: Preparing LEF files for the max corner...
[INFO]: Running linter (Verilator) (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/synthesis/linter.log)...
[INFO]: 0 errors found by linter
[WARNING]: 49 warnings found by linter
[STEP 1]
[INFO]: Running Synthesis (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/synthesis/1-synthesis.log)...
[STEP 2]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/synthesis/2-sta.log)...
[STEP 3]
[INFO]: Running Initial Floorplanning (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/floorplan/3-initial_fp.log)...
[INFO]: Floorplanned with width 2351.52 and height 2350.08.
[STEP 4]
[INFO]: Running IO Placement...
[STEP 5]
[INFO]: Running Tap/Decap Insertion (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/floorplan/5-tap.log)...
[INFO]: Power planning with power {VPWR} and ground {VGND}...
[STEP 6]
[INFO]: Generating PDN (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/floorplan/6-pdn.log)...
[STEP 7]
[INFO]: Running Global Placement (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/placement/7-global.log)...
[STEP 8]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/placement/8-gpl_sta.log)...
[STEP 9]
[INFO]: Running Placement Resizer Design Optimizations (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/placement/9-resizer.log)...
[STEP 10]
[INFO]: Running Detailed Placement (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/placement/10-detailed.log)...
[STEP 11]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/placement/11-dpl_sta.log)...
[STEP 12]
[INFO]: Running Clock Tree Synthesis (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/cts/12-cts.log)...
[STEP 13]
[INFO]: Running Single-Corner Static Timing Analysis (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/cts/13-cts_sta.log)...
[STEP 14]
[INFO]: Running Placement Resizer Timing Optimizations (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/cts/14-resizer.log)...
[STEP 15]
[INFO]: Running Global Routing Resizer Design Optimizations (log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/routing/15-resizer_design.log)...
[ERROR]: during executing openroad script /openlane/scripts/openroad/resizer_routing_design.tcl
[ERROR]: log: designs/soc_top/runs/RUN_2026.08.26_05.09.32/logs/routing/15-resizer_design.log
[ERROR]: Last 10 lines:
[INFO GRT-0101] Running extra iterations to remove overflow.
[INFO GRT-0103] Extra Run for hard benchmark.
[INFO GRT-0197] Via related to pin nodes: 768443
[INFO GRT-0198] Via related Steiner nodes: 31983
[INFO GRT-0199] Via filling finished.
[INFO GRT-0111] Final number of vias: 1045011
[INFO GRT-0112] Final usage 3D: 5181055
[ERROR GRT-0119] Routing congestion too high. Check the congestion heatmap in the GUI and load /openlane/designs/soc_top/runs/RUN_2026.08.26_05.09.32/tmp/routing/resize
n-routing-design-congestion.rpt in the DRC viewer.
Error: resizer_routing_design.tcl, 46 GRT-0119
Child process exited abnormally

[ERROR]: Creating issue reproducible...
[INFO]: Saving runtime environment...
OpenLane TCL Issue Packager

EFABLESS CORPORATION AND ALL AUTHORS OF THE OPENLANE PROJECT SHALL NOT BE HELD
LIABLE FOR ANY LEAKS THAT MAY OCCUR TO ANY PROPRIETARY DATA AS A RESULT OF USING
THIS SCRIPT. THIS SCRIPT IS PROVIDED ON AN "AS IS" BASIS, WITHOUT WARRANTIES OR
CONDITIONS OF ANY KIND.

BY USING THIS SCRIPT, YOU ACKNOWLEDGE THAT YOU FULLY UNDERSTAND THIS DISCLAIMER
AND ALL IT ENTAILS.

Parsing config file(s)-
Setting up /openlane/designs/soc_top/runs/RUN_2026.08.26_05.09.32/issue_reproducible..
[WARN] /openlane/designs/soc_top/runs/RUN_2026.08.26_05.09.32/tmp/15-soc_top.sdc was not found, might be a product. Skipping
Done.
[INFO]: Reproducible packaged: Please tarball and upload 'designs/soc_top/runs/RUN_2026.08.26_05.09.32/issue_reproducible' if you're going to submit an issue.
[ERROR]: Step 15 (routing) failed with error:
code 1 -level 0 -errorcode NONE -errorinfo {
  while executing
  "throw_error"
  (procedure "run_tcl_script" line 219)
  invoked from within
  "run_tcl_script -tool openroad -no consume {*} $args"
  (procedure "run_openroad_script" line 2)
  invoked from within
  "run_openroad_script ${:env(SCRIPTS_DIR)}/openroad/resizer_routing_design.tcl -indexed_log $log -save "dir=${:env(routing_tmpfiles)},def,sdc,odb,netlist,..."

```

Figure 18. OpenLane physical-design flow execution and routing congestion error.

8.4 Why Routing Congestion Occurred

The earlier generic synthesis showed approximately 70,933 cells for axi_ram, including about 32,768 flip-flops and 32,840 multiplexers. Implementing a comparatively large RAM as standard-cell logic creates high placement density and routing demand. The OpenLane run reported approximately 1,045,011 final vias and then reported 'Routing congestion too high'.

This is an implementation limitation, not a functional failure of the SoC. The functional simulation and the pre-routing physical-design stages completed successfully.

8.5 What Would Improve the Physical Implementation

- Use an SRAM macro rather than a large bank of flip-flops/muxes for a realistic memory implementation.
- Reduce RAM size for a smaller demonstration chip, if architecturally acceptable.
- Experiment with utilization/floorplan settings to provide more routing space.
- Repeat routing and signoff after memory/physical-design optimization.

9. Results, Limitations and Future Work

Milestone	Status
UART TX/RX module verification	Completed
AXI UART verification	Completed
AXI RAM verification	Completed
AXI interconnect verification	Completed
Complete RISC-V SoC simulation	Passed
GTKWave analysis	Completed
Yosys generic synthesis	Completed
Synthesized schematic	Generated
OpenLane synthesis → CTS	Completed
Routing	Stopped by congestion
Final routed GDS	Not generated / not claimed

9.1 Limitations

- Current RAM implementation is physically expensive when mapped to standard cells.
- Routing congestion prevented completion of the full physical-design flow.
- Final DRC/LVS and routed GDS are therefore outside the current result set.

9.2 Future Work

- Introduce a suitable SRAM macro or more area-efficient memory implementation.
- Repeat OpenLane physical implementation after memory optimization.
- Complete routing, DRC/LVS and GDS generation.
- Add additional firmware tests and peripherals if required.

9.3 Conclusion

The project demonstrates an end-to-end digital design path from RISC-V firmware and CPU execution through AXI transactions, memory-mapped UART control, functional verification and ASIC physical-design exploration. Its strongest evidence is the verified CPU→AXI→UART transaction together with synthesis and OpenLane implementation results.

Attribution: The PicoRV32 processor core used in this project is open-source hardware developed and maintained through the YosysHQ PicoRV32 project. It was integrated as pre-existing IP and was not developed from scratch as part of this project.

References

1. YosysHQ, *PicoRV32 – A Size-Optimized RISC-V CPU*. GitHub. Available at: <https://github.com/YosysHQ/picorv32>.
(Open-source PicoRV32 processor core integrated into this project.)
2. Arm Ltd., *AMBA AXI and ACE Protocol Specification – AXI4, AXI4-Lite, ACE and ACE-Lite*. Available at: <https://documentation-service.arm.com/static/67ab5a186dbc975ccea91734>.
3. YosysHQ, *Yosys Open SYNthesis Suite*. Available at: <https://yosyshq.net/yosys/>
4. Efabless, *OpenLane – Open Source RTL-to-GDSII Flow*. Available at: <https://github.com/The-OpenROAD-Project/OpenLane>
5. OpenROAD Project, *OpenROAD – Open Source RTL-to-GDS Implementation System*. Available at: <https://github.com/The-OpenROAD-Project/OpenROAD>
6. SkyWater Technology and Google, *SKY130 Open Source PDK*. Available at: <https://github.com/google/skywater-pdk>
7. RISC-V International, *The RISC-V Instruction Set Architecture*. Available at: <https://riscv.org/technical/specifications/>